LEAN ALGEBRAIC JACOBIAN FORMALIZATION

Complete Recovery and Execution Plan

AJCR-first strategy for Picard representability, Galois descent, and final Jacobian construction

Version: 1.0
Date: August 2026
Primary endpoint: Kernel-checked arbitrary-field Pic^0 representability
Secondary endpoint: JacobianData constructed from the same pinned representation

Working principle

Progress is measured by closed theorem dependencies, consumed producers, deleted obsolete paths, and kernel-checked endpoints - not by raw LOC growth.

Contents

1. Executive summary
2. The three principal issues
3. Binding architecture decision
4. Definition of success
5. Phase 0 - Freeze, measure, and establish the critical root
6. Phase 1 - Lock the endgame contract and route guards
7. Phase 2 - Cap the parameter-decoupling lane
8. Phase 3 - Repair type and instance mismatches
9. Phase 4 - Construct the rank-one Picard chart
10. Phase 5 - Cover over a separably closed field
11. Phase 6 - Produce Pic^0 over the separable closure
12. Phase 7 - Complete finite Galois descent
13. Phase 8 - Construct the final Jacobian witness
14. AJC integration strategy
15. Eight-round execution schedule
16. Agent roles and dispatch specifications
17. Merge, credit, and stop rules
18. Progress dashboard and acceptance tests
19. Risk register and fallback decisions
20. Immediate action list

Appendix A. Suggested theorem signatures

Appendix B. Minimal dependency graph

1. Executive summary

The project has made real progress, especially in widened divisor-family infrastructure, rank calculations, and finite-Galois descent machinery. However, the current code growth does not yet correspond to comparable movement toward the final Picard representability theorem. The central reason is architectural: large divisor degree gives coverage but destroys Abel-map injectivity, while genus degree gives rank-one uniqueness but is harder to cover over an arbitrary base field.

The recommended route is therefore AJCR-first and follows the curve-specialized classical construction: build a rank-one Picard open over a separably closed field, represent it by an open inside the degree-g divisor scheme, cover the Picard functor by translations of this open, then descend the resulting Pic^0 through finite Galois descent. AJC should be used as the shared descent engine and long-term infrastructure project rather than as a second independent Picard proof.

> **Primary decision**
>
> The next endpoint-critical theorem is the family-level natural isomorphism between the rank-one open inside Div^g and the rank-one open inside the Picard functor. Parameter decoupling remains useful infrastructure, but it is no longer the main Picard endpoint lane.

The complete execution plan below is organized around one dependency chain:

```
PicRankOneOpen
    -> rankOneAbelIso
    -> rankOne_translate_cover_sepClosed
    -> pic0_sepClosed_representableBy
    -> finiteGaloisDescent
    -> pic0_representableBy
    -> JacobianData
```

2. The three principal issues

2.1 Coverage degree and injectivity degree are incompatible

On a genus-g curve, a nonspecial divisor of degree n has h^0 = n + 1 - g. At a sufficiently large admissible degree, coverage is available, but h^0 is greater than 1 in positive genus. The Abel map then has positive-dimensional linear-system fibres and cannot directly be the monomorphism or open immersion required by the existing Picard chart seam.

At degree n = g, the same rank formula gives h^0 = 1 on the nonspecial locus, so uniqueness and an open-immersion chart become possible. However, the required degree-g translation data need not exist over an arbitrary base field. The project has therefore been attempting to make two different parameter regimes solve one seam.

Parameter regime	What it gives	What fails
n = g	Rank-one fibres; potential open immersion	Pointwise coverage over the original base field
n = admissible > g	Uniform coverage and large-degree control	Injectivity; the Abel fibres have h^0 > 1

2.2 Lean interfaces are hiding mathematical misalignment

Several classifier layers use one symbol for distinct semantic roles: curve genus, divisor degree, and window parameters. Equivalent geometric objects are also represented by different Lean expressions, causing expensive definitional-equality and typeclass-search failures. Weak existential witnesses and source/test-locus mismatches have allowed locally true lemmas to accumulate without closing the intended producer.

- Hard-coded diagonal relation n = g in classifier and RepresentableBy layers.
- Repeated transports between relCurve-style and explicit base-change expressions.
- Duplicate diagonal and generalized proofs surviving in parallel.
- Fieldwise uniqueness results used where the consumer needs a natural inverse on arbitrary test schemes.
- Compatibility wrappers that move hypotheses but do not create the missing geometric map.

2.3 The final theorem chain is not yet closed

The repository contains many strong components, but the final path is still discontinuous. The missing central theorem is not another rank or coverage lemma; it is a family-level identification of the correct divisor open with the correct Picard open, followed by a complete descent theorem returning both the descended scheme and its RepresentableBy certificate.

> **Diagnostic rule**
>
> A theorem is endpoint progress only when it is root-reachable and consumed by the next declaration on the critical path. An unconsumed alias, wrapper, generalized lemma, or isolated proof is infrastructure inventory.

3. Binding architecture decision

3.1 Chosen route

Use the curve-specialized rank-one atlas over a separably closed field, then descend. The binding route is:

```
rank-one open inside Div^g
    <-> rank-one Picard open
    -> translated open cover over k^s
    -> Pic^0 represented over k^s
    -> finite Galois descent
    -> Pic^0 represented over k
```

3.2 Role of the high-degree divisor lane

Complete one useful vertical arbitrary-degree representability spine, then freeze broad propagation unless a named downstream theorem consumes it. This work remains valuable for divisor geometry, Brill-Noether theory, projective-bundle descriptions, and a possible quotient fallback, but it should not be counted as direct completion of the old Picard open-chart seam.

3.3 When a quotient route may be reconsidered

A large-degree Abel quotient should be reopened only after a non-circular Lean-shaped theorem contract is written. The contract must construct a scheme quotient, prove its universal property, prove base-change compatibility, and produce the desired map to the Picard functor without assuming Picard representability. Merely defining equality of induced Picard class is not sufficient.

4. Definition of success

The project is complete only when the same pinned representability datum is consumed all the way to the Jacobian witness.

```
noncomputable def pic0_representableBy :
    Sigma fun J : Scheme => (pic0Functor C).RepresentableBy J

noncomputable def jacobianData :
    JacobianData C
```

The final verification must include:

- Both declarations imported by the narrow critical-path root and the full project root.
- Axiom audit showing no project-specific axioms or sorryAx.
- Group structure, universal element, base change, and cocycle coherence derived from the same representation rather than separately chosen objects.

- AJC either consumes this representability engine or formally derives its own producer from the shared implementation.

5. Phase 0 - Freeze, measure, and establish the critical root

5.1 Pin reproducible revisions

1. Pin the last commit before the major LOC increase.
2. Pin current main.
3. Pin the latest committed state of parameter-decoupling run 0108.
4. Pin the latest committed state of rank/index run 0109.

5.2 Build the baseline ledger

For each revision, measure the items below. The numbers must be generated from scripts and recorded with the exact commit hash.

Measurement	Purpose
Root-reachable modules	Exclude dead scratch lanes from progress claims
Root-reachable sorry/axioms	Measure kernel status on the actual path
Old DivFamZar consumers	Verify that migration removes the old architecture
New DivFamZarAff consumers	Measure adoption outside the implementation cone
Critical-path declarations	Measure theorem movement rather than file growth
Heartbeat/depth overrides	Detect proof-term and elaboration deterioration
Critical-root time and peak memory	Measure iteration speed
Full-root build status	Catch duplicate declarations and hidden imports

5.3 Create a narrow critical-path root

```
-- Picard/Pic0CriticalPath.lean
import ...DivRepChartClassUnivAffRepresentable
import ...Pic0ChartLocusH0Rank
import ...Pic0RankOneOpen
import ...Pic0RepresentabilityFinal

#check divFunctorAff_genus_representableBy
#check rankOneAbelIso
#check pic0_sepClosed_representableBy
#check pic0_representableBy
```

5.4 Acceptance criteria

- The baseline can be reproduced by another person from the pinned commits.
- Every endpoint declaration is checked in the narrow root.
- A theorem proved in isolation receives no critical-path credit until imported here.

6. Phase 1 - Lock the endgame contract and route guards

6.1 Create the contract file

Create Picard/Pic0EndgameContract.lean. The statements must elaborate even if some proofs temporarily depend on explicit local hypotheses. Every declaration must list its exact mathematical source, Lean prerequisites, and immediate consumer.

```
def PicRankOneOpen : ...
def DivRankOneOpen : ...
def rankOneAbel : ...
def divisorOfRankOne : ...

theorem rankOneAbelIso : ...
theorem rankOne_translate_cover_sepClosed : ...

noncomputable def pic0_sepClosed_representableBy : ...
noncomputable def pic0_representableBy : ...
```

6.2 Add a route guard against the false high-degree seam

Assemble a root-imported negative theorem showing that, in positive genus and at parameter $n > g$, the unrestricted Abel map cannot be the desired open immersion because its fibres have at least two sections. This theorem prevents future agents from accidentally treating the admissible divisor producer as direct completion of the old chart seam.

6.3 Acceptance criteria

- The endpoint appears as one short dependency graph with no hidden route fork.
- High-degree divisor representability is clearly labeled infrastructure.
- Any quotient proposal must first replace the binding architecture decision with a complete theorem contract.

7. Phase 2 - Cap the parameter-decoupling lane

7.1 Separate semantic parameters

Name	Meaning	Permitted use
gC	Genus of the curve	Euler characteristic, Riemann-Roch, cohomological bounds
nDiv	Degree represented by the divisor functor	DivFamZarAff, certified families, divisor scheme degree
mWin	Lower window index	Window construction only
sWin	Window width	Window construction only
r1, r2	Ranks of section modules	Bases and Grassmannian data
m, Z	Abel translation/index data	Chart value and degree calibration

7.2 Complete one vertical spine

Work top-down from the desired producer. Generalize only declarations reached by type errors on one complete path; do not pre-generalize the entire library.

```
noncomputable def divFunctorAff_representableBy_at
    (gC nDiv : Nat)
    (hchi : chi O_C = 1 - gC) :
    Sigma fun D => (divFunctorAff C nDiv).RepresentableBy D
```

Then define both the genus and admissible producers as specializations. The old diagonal proof must be deleted or reduced to a one-line specialization.

7.3 Mandatory regression tests

- Diagonal test: nDiv = gC.
- Off-diagonal test: nDiv = admissibleCoverageParameter C.
- Next-consumer test: the generalized producer is actually used by a downstream theorem.

7.4 Stop condition

After the arbitrary-degree producer lands, freeze broad parameter propagation unless a named theorem consumes it. Do not spend further rounds generalizing unused layers.

8. Phase 3 - Repair type and instance mismatches

8.1 Canonicalize base-changed curves

Choose one public spelling, preferably relCurve C L, throughout the critical path. Place all identifications with explicit tensor/base-change expressions in one bridge module.

```
-- Curve/RelativeCurveBridge.lean
-- canonical isomorphisms
-- transported instances
-- inferInstanceAs bridges
-- base-change simp lemmas
```

8.2 Build minimal performance regressions

Isolate each identity-like application that currently requires extreme heartbeat or recursion settings. The refactor is complete only when the reproducer compiles under the normal budget.

8.3 Add semantic anti-vacuity tests

- Every existential witness must explicitly recover or represent the input Picard class.
- A zero or unrelated divisor must not satisfy the conclusion accidentally.
- Field-level uniqueness must not be credited as a natural inverse on arbitrary test schemes.

8.4 Acceptance criteria

- Critical files use one public curve spelling.
- Casts and instance transports are concentrated in the bridge module.
- No new parallel high-priority instances are introduced.

- The regression examples compile without enlarged global limits.

9. Phase 4 - Construct the rank-one Picard chart

9.1 Define one public rank-one Picard open

For a test scheme T and f_T : C_T -> T, define the open subfunctor consisting of line bundles L such that R^1 f_* L = 0 and f_* L is locally free of rank one, with the required base-change compatibility. This should be the single public notion of the good Picard locus.

```
def PicRankOneOpen :
    Subfunctor (picFunctor C)
```

Existing chartLocus, H0Rank, and H1-vanishing predicates must become implementation lemmas or be proved equivalent to this definition.

9.2 Define the corresponding open inside Div^g

Use the universal divisor on the existing genus-degree representer to define the maximal open W on which its associated line bundle lies in PicRankOneOpen.

```
def DivRankOneOpen : Scheme
def rankOneAbel :
    yoneda.obj (DivRankOneOpen C) -> PicRankOneOpen C
```

9.3 Construct the inverse canonically

Given L in PicRankOneOpen(T), set N = f_* L. Use the canonical evaluation map f^*N -> L. Equivalently, this is a canonical section of L tensor f^*(N^{-1}). Its zero scheme is the desired relative effective Cartier divisor. Do not choose a generator of N.

Prove that the zero scheme is finite locally free of degree g, commutes with arbitrary base change, and defines the same relative Picard class because O(D) and L differ only by a pullback from T.

```
def divisorOfRankOne :
    PicRankOneOpen C -> yoneda.obj (DivRankOneOpen C)

theorem rankOneAbel_divisorOfRankOne : ...
theorem divisorOfRankOne_rankOneAbel : ...

def rankOneAbelIso :
    yoneda.obj (DivRankOneOpen C) ≅ PicRankOneOpen C
```

9.4 Obtain the open immersion immediately

Compose the isomorphism h_W ~= PicRankOneOpen with the open inclusion PicRankOneOpen -> Pic. This gives the family-level open immersion needed by the final representability criterion.

```
theorem rankOneAbel_isOpenImmersion :
    IsOpenImmersion.presheaf (rankOneAbelToPic C)
```

9.5 Cleanup requirement

- Delete or demote generator-selection constructions.
- Demote fieldwise h^0 = 1 uniqueness to corollaries.
- Delete source/test coupling wrappers no longer consumed.
- Keep only lemmas used by rankOneAbelIso or its openness proof.

> **Main milestone**
>
> The theorem rankOneAbelIso is the decisive architectural milestone. It converts fibrewise intuition into a natural family-level chart and should cause net deletion of workaround code.

10. Phase 5 - Cover over a separably closed field

10.1 Translation theorem

Assume k is separably closed. For every extension K/k and every Picard class lambda on C_K, construct a line bundle L0 on C/k such that the translated class has h^0 = 1 and h^1 = 0. The base-field origin of L0 is essential because each translated open must itself be defined over k.

10.2 Formal proof strategy

5. Twist by a sufficiently positive base-field line bundle until H^1 vanishes and H^0 is nonzero.
6. If h^0 is greater than one, choose a k-rational point at which a nonzero section does not vanish.
7. Subtract that point and use the exact sequence to lower h^0 by one while preserving H^1 = 0.
8. Repeat until h^0 = 1.
9. Return membership of the translated input class in PicRankOneOpen.

```
theorem exists_translation_mem_picRankOneOpen
    [IsSepClosed k]
    (K : Type*) [Field K] [Algebra k K]
    (lam : picFunctor C (Spec K)) :
    exists L0 : LineBundle C,
      translate L0 lam in PicRankOneOpen C (Spec K)
```

10.3 Acceptance criteria

- The output is explicitly tied to the input class lam.
- The theorem plugs directly into the coverage input of the final gluing criterion.
- No unrelated witness structure is introduced.

11. Phase 6 - Produce Pic^0 over the separable closure

11.1 Cover the Picard functor by translated rank-one opens

For every base-field line bundle L0, translate PicRankOneOpen. Translation preserves representability by open subschemes. The separably closed translation theorem shows that these opens cover field-valued points after arbitrary extensions.

11.2 Apply the open-cover representability criterion

```
noncomputable def pic_sepClosed_representableBy
    [IsSepClosed k] :
    Sigma fun P : Scheme => (picFunctor C).RepresentableBy P

noncomputable def pic0_sepClosed_representableBy
    [IsSepClosed k] :
    Sigma fun J : Scheme => (pic0Functor C).RepresentableBy J
```

11.3 Verification

```
#check pic0_sepClosed_representableBy
#print axioms pic0_sepClosed_representableBy
```

- Build the narrow critical root.
- Build the full AJCR root.
- Record build time, peak memory, and axiom output in the progress ledger.

> **First headline endpoint**
>
> A kernel-checked pic0_sepClosed_representableBy is the first genuine headline milestone. Before this theorem exists, all rank-one work remains local infrastructure.

12. Phase 7 - Complete finite Galois descent

12.1 Package the geometric descent input

Use the degree-g Abel map and group structure to prove that the separably closed representative is finite type, proper, and geometrically irreducible in the required component. Package these properties and the universal element in one structure rather than leaving disconnected instances.

```
structure Pic0SepClosedDescentInput where
  J : SchemeOver ks
  represents : ...
  finiteType : ...
  proper : ...
  groupStructure : ...
  geomIrreducible : ...
  universalElement : ...
```

12.2 Descend to a finite Galois extension

10. Descend finite-presentation data and the universal element from k^s to a finite separable extension L/k.
11. Enlarge L to a finite Galois extension.
12. Use uniqueness of representing objects to obtain the semilinear Galois action.
13. Prove the cocycle condition by uniqueness.
14. Produce a Galois-stable affine cover; use the irreducible group structure to supply orbit-in-affine conditions.
15. Form affine invariant quotients, prove overlap compatibility, and glue.

16. Descend the universal element and verify the descended Yoneda isomorphism.

12.3 Reuse the AJC descent engine

Port or share the existing AJC finite-Galois quotient, stable affine cover, overlap, and gluing modules. Do not implement a second parallel descent stack in AJCR.

```
theorem representableBy_of_finiteGalois_baseChange
    (J_L : SchemeOver L)
    (hrep : (F.baseChange L).RepresentableBy J_L)
    (rho : SemilinearGaloisAction J_L)
    (hcocycle : rho.Cocycle)
    (haffine : rho.OrbitsLieInAffineOpens) :
    Sigma fun J : SchemeOver k => F.RepresentableBy J
```

12.4 Produce arbitrary-field Pic^0

```
noncomputable def pic0_representableBy :
    Sigma fun J : Scheme => (pic0Functor C).RepresentableBy J

#check pic0_representableBy
#print axioms pic0_representableBy
```

12.5 Acceptance criteria

- The descent theorem returns both the descended scheme and its RepresentableBy certificate.
- It is consumed immediately by pic0_representableBy.
- The theorem is shared with or imported from AJC, not duplicated.

13. Phase 8 - Construct the final Jacobian witness

13.1 Consume the same pinned representation

```
noncomputable def jacobianData :
    JacobianData C
```

Construct all remaining structures from the same pic0_representableBy datum: the group object, identity component, universal element, base-change isomorphism, and cocycle coherence. Avoid independently chosen representatives or repeated applications of classical choice.

13.2 Downstream order

17. Identify the degree-zero identity component.
18. Construct and verify the group-object structure.
19. Prove finite type, properness, smoothness, and dimension g.
20. Construct the Abel-Jacobi map.
21. Prove the Albanese universal property.
22. Prove field-base-change compatibility and cocycle coherence.
23. Audit all protected challenge declarations.

13.3 Final verification

```
#check jacobianData
#print axioms jacobianData
```

- No project-specific axioms or sorryAx.
- Both the narrow root and full project root build.
- All final structures are derived from the same representation.

14. AJC integration strategy

14.1 Immediate role of AJC

AJC should focus on shared descent, quotient/gluing, and general substrate. It should not simultaneously pursue a second independent FGA Picard proof while AJCR is completing the curve-specialized route.

14.2 Permitted AJC work before AJCR completion

- Import and performance hygiene that lowers the cost of every subsequent proof.
- Finite-Galois quotient and gluing declarations with a named AJCR consumer.
- Substrate that can be ported directly between the two projects.
- Removal of false degree conventions and duplicated Picard carriers.

14.3 Port after AJCR succeeds

24. Move reusable rank-one/open-chart lemmas into shared modules.
25. Move or share the finite-Galois descent capstone.
26. Instantiate AJC Picard infrastructure with the AJCR representability theorem.
27. Replace the central AJC FGA representability placeholder.
28. Retain broader FGA/Quot developments only as optional general infrastructure.

15. Eight-round execution schedule

Round	Focus	Deliverable	Acceptance gate
1	Audit and route guard	Pinned baseline, critical root, negative high-degree Abel guard	No production expansion
2	Endgame contract and interface cleanup	Elaborating theorem contract; relCurve bridge; performance regressions	Contract is root-imported
3	Rank-one public loci	PicRankOneOpen and DivRankOneOpen with openness and base change	One public good-locus API
4	Canonical family inverse	divisorOfRankOne and rankOneAbelIso	Arbitrary-test natural isomorphism
5	Separably closed coverage	Translation theorem reducing h^0 to one while preserving H^1 = 0	Direct input to gluing criterion
6	Separably closed producer	pic0_sepClosed_representable By plus axiom audit	First headline milestone

Round	Focus	Deliverable	Acceptance gate
7	Descent input and finite-level action	Finite type/proper/group package; finite Galois action and stable affine cover	Exact input to descent capstone
8	Descent capstone and Jacobian wiring	pic0_representableBy and jacobianData	Kernel-checked arbitrary-field endpoint

The capped parameter-decoupling spine may run in parallel during rounds 1-2, but it must not edit the public rank-one API or delay rounds 3-6.

16. Agent roles and dispatch specifications

16.1 Mathematical route reviewer

Read-only. Verifies theorem statements against the mathematical source, searches for counterexamples, checks circularity, and classifies every blocker as mathematical, architectural, type-theoretic, or performance-related.

16.2 Interface auditor

Creates minimal failing harnesses, parameter-role maps, and performance regressions. It does not write broad production proofs or introduce new public APIs.

16.3 Vertical-slice builder

Owns one complete producer-to-consumer edge. A builder is not assigned a topic such as "work on Picard"; it is assigned a theorem whose output is consumed by a named next theorem.

16.4 Merge and kernel reviewer

Checks root reachability, duplicate declarations, obsolete paths, axiom output, import cost, heartbeats, and whether the new theorem is actually consumed.

16.5 Public API owner

One human or designated agent owns the public theorem signatures. Multiple agents may prove local lemmas, but they may not create competing top-level interfaces.

16.6 Recommended dispatch texts

> **Dispatch A - audit**
>
> Pin revisions, produce the critical-path ledger, identify the exact next consumer of every claimed Picard theorem, and install the high-degree route guard. Do not add production infrastructure.

> **Dispatch B - capped decoupling**
>
> Complete one arbitrary-degree RepresentableBy spine. The diagonal theorem must become a specialization; both genus and admissible parameters must compile; duplicated diagonal implementations must be deleted.

> **Dispatch C - canonical inverse**
>
> Construct divisorOfRankOne from the evaluation morphism on arbitrary test schemes and produce rankOneAbelIso. No generator choice is permitted; the theorem must be consumed by the open-immersion proof.

> **Dispatch D - descent capstone**
>
> Reuse the AJC finite-Galois quotient/gluing engine and return both the descended scheme and RepresentableBy certificate. Then consume it in pic0_representableBy.

17. Merge, credit, and stop rules

17.1 Critical-path credit

A commit receives endpoint credit only when all conditions below hold:

29. It adds or completes a theorem in the endgame contract.
30. The theorem is imported by the narrow critical root.
31. Its immediate downstream consumer uses it.
32. The relevant module kernel-builds.
33. Axiom output is recorded.
34. The obsolete implementation is deleted or reduced to a specialization.

17.2 Zero-credit work

- Aliases, wrappers, and compatibility declarations with no consumer.
- Unused _at lemmas and root-unreachable modules.
- Fieldwise statements when the endpoint requires arbitrary test schemes.
- A quotient object without an effective universal property.
- A witness not explicitly related to the theorem input.
- New global heartbeat, depth, or synthesis limits.

17.3 Stop and redesign conditions

- Two consecutive merges add code without closing a contract theorem.
- Old and new carrier implementations continue to grow simultaneously.
- A trivial application requires a new large heartbeat or recursion setting.
- Three or more repeated transports reconcile the same two types.
- A field-level theorem is being used to justify a family-level natural transformation.
- A quotient route is proposed without a complete, non-circular theorem contract.
- An existential theorem can be satisfied by an object unrelated to its input.

18. Progress dashboard and acceptance tests

18.1 Dashboard categories

Category	Definition	Score
Green	Root-reachable producer consumed	Full credit

Category	Definition	Score
	by the next critical-path declaration	
Yellow	Necessary prerequisite with a named, typechecked consumer	Partial credit
Red	Alias, wrapper, dead lane, duplicate proof, larger kernel limit, or unconsumed API	No or negative credit

18.2 Required metrics per round

Metric	Desired direction
Critical-path edges closed	Increase
Root-reachable producers consumed	Increase
Old carrier consumers	Decrease
Duplicate proof bodies	Decrease
Compatibility-only declarations	Decrease or remain zero
Root-unreachable files added	Zero
Heartbeat/depth overrides	Zero new; ideally decrease
Critical-root build time and memory	Stable or improve
Full-root build	Pass at every checkpoint
Final producer axioms	No sorryAx

18.3 Core acceptance tests

```
#check rankOneAbelIso
#check rankOneAbel_isOpenImmersion
#check exists_translation_mem_picRankOneOpen
#check pic0_sepClosed_representableBy
#check representableBy_of_finiteGalois_baseChange
#check pic0_representableBy
#check jacobianData
```

19. Risk register and fallback decisions

ID	Risk	Likelihood	Impact	Mitigation
R1	Evaluation-map zero scheme requires missing relative Cartier-divisor infrastructure	Medium	High	Isolate exact missing lemmas; implement only the curve-specific statement; avoid generator-based workaround
R2	Rank-one locus API does	High	Medium	Rewrite the seam

ID	Risk	Likelihood	Impact	Mitigation
	not match existing final seam			around the natural isomorphism rather than add coupling wrappers
R3	Separably closed translation theorem needs unavailable cohomology/point-selection lemmas	Medium	Medium	Formalize a minimal curve-specific induction; keep output directly tied to PicRankOneOpen
R4	Finite Galois quotient capstone is incomplete	High	High	Reuse AJC affine pieces; make global quotient/gluing one shared theorem with AJCR as integration test
R5	Kernel proof terms become too large	High	Medium	Narrow roots, precise imports, vertical modules, specialization rather than duplicated proofs
R6	Agents reopen the high-degree direct Abel seam	Medium	High	Root-imported negative route guard and binding contract
R7	Rank-one route fails due to a genuine descent obstruction	Low/Medium	High	Only then reopen the quotient route after a complete non-circular theorem contract

19.1 Fallback hierarchy

35. First fallback: adjust the rank-one chart index or locus while preserving the canonical evaluation-map inverse.
36. Second fallback: strengthen shared finite-Galois descent infrastructure.
37. Third fallback: reopen a curve-specific effective quotient theorem, but only after proving its contract is non-circular.
38. Do not return to broad, unconstrained FGA scaffolding as the immediate headline route.

20. Immediate action list

20.1 First three actions

39. Create the pinned baseline and narrow critical root.
40. Install the endgame contract and high-degree route guard.
41. Dispatch the canonical rank-one inverse spike while a separate bounded lane completes the arbitrary-degree divisor producer.

20.2 First decision checkpoint

After the rank-one inverse spike, make a binding decision based on a concrete Lean result:

- If rankOneAbelIso elaborates on arbitrary tests, proceed immediately to separably closed coverage and final assembly.

- If it fails because of a precisely identified missing curve-specific theorem, add only that theorem to the plan.
- If it fails because the public Picard or divisor interfaces are fundamentally wrong, repair those interfaces before any further proof expansion.
- Only if the mathematical construction itself fails should the quotient route be reopened.

20.3 Final concise summary

> **Execution summary**
>
> Measure first. Lock one theorem-shaped route. Finish one capped arbitrary-degree spine. Repair canonical types. Prove the family-level rank-one divisor/Picard isomorphism. Cover over the separable closure. Reuse AJC for finite Galois descent. Construct Pic^0 and JacobianData from one pinned representation.

Appendix A. Suggested theorem signatures

A.1 Arbitrary-degree divisor producer

```
noncomputable def divFunctorAff_representableBy_at
    (gC nDiv : Nat)
    (hchi : Sheaf.chi (...) = 1 - (gC : Int)) :
    Sigma fun D => (divFunctorAff C nDiv).RepresentableBy D
```

A.2 Rank-one loci

```
def PicRankOneOpen : Subfunctor (picFunctor C)
def DivRankOneOpen : Scheme

theorem picRankOneOpen_baseChange : ...
theorem picRankOneOpen_isOpen : ...
```

A.3 Canonical chart isomorphism

```
def rankOneAbel :
    yoneda.obj (DivRankOneOpen C) -> PicRankOneOpen C

def divisorOfRankOne :
    PicRankOneOpen C -> yoneda.obj (DivRankOneOpen C)

def rankOneAbelIso :
    yoneda.obj (DivRankOneOpen C) ≅ PicRankOneOpen C

theorem rankOneAbel_isOpenImmersion : ...
```

A.4 Separably closed coverage and representation

```
theorem exists_translation_mem_picRankOneOpen
```

```
    [IsSepClosed k] (...) : ...

noncomputable def pic0_sepClosed_representableBy
    [IsSepClosed k] :
    Sigma fun J => (pic0Functor C).RepresentableBy J
```

A.5 Galois descent and final endpoint

```
theorem representableBy_of_finiteGalois_baseChange (...) :
    Sigma fun J : SchemeOver k => F.RepresentableBy J

noncomputable def pic0_representableBy : ...
noncomputable def jacobianData : JacobianData C
```

Appendix B. Minimal dependency graph

```
Div^g represented
      |
      v
DivRankOneOpen -----------
      |                    |
      | rankOneAbelIso     | canonical evaluation divisor
      v                    |
PicRankOneOpen <-----------
      |
      | translated opens cover over k^s
      v
pic0_sepClosed_representableBy
      |
      | finite type + proper + group + descent datum
      v
representableBy_of_finiteGalois_baseChange
      |
      v
pic0_representableBy
      |
      v
JacobianData
```

Everything outside this graph is either prerequisite infrastructure, reusable general theory, or a fallback. It should not be allowed to obscure whether the endpoint chain is actually closing.

End of plan